



Processamento e Otimização de Consultas

Banco de Dados

Prof. Dr. Carlos Melo

 carlos.melo@upe.br



Processamento e Otimização de Consultas

- O SQL que escrevemos não é executado diretamente no banco
- Antes:
 - O banco analisa a consulta
 - escolhe estratégias
 - busca o menor custo possível



Processamento e Otimização de Consultas

- O banco interpreta a consulta e a transforma em operações internas
- Depois escolhe um **plano de execução**

```
1 SELECT nome  
2 FROM empregado  
3 WHERE salario > 3000;
```



Processamento e Otimização de Consultas

- Ao todo três etapas são executadas

Etapa	Função
parser	Interpreta o SQL
otimizador	Escolhe uma Estratégia
executor	Executa a Consulta

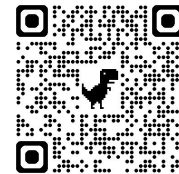
- Duas consultas equivalentes podem possuir desempenhos diferentes.



Álgebra Relacional

- É o modelo formal de operações relacionais que delimita o "passo a passo" matemático por baixo dos panos de uma consulta SQL. Ela representa:
 - Operações sobre tabelas
 - Manipulação de Conjuntos
 - Processamento interno das consultas
- Quando rodamos uma consulta, o banco converte o SQL em Álgebra Relacional para descobrir a forma mais rápida de executar a tarefa.

Álgebra Relacional



- Principais Operações
 - Seleção (σ)
 - Projeção (π)
 - Junção ($\bowtie$)
 - União ($\cup$)
 - Diferença ($-$)
 - Produto Cartesiano ($\times$)



Seleção (σ)

- Filtra linhas de uma relação
 - Utiliza uma condição lógica
 - Ex: σ salario > 3000 (**EMPREGADO**)

-> retorna apenas empregados com salário maior que 3000



```
1 SELECT nome
2 FROM empregado
3 WHERE salario > 3000;
```

id	nome	salario	departamento
1	Ana	5000	TI
3	Maria	7000	TI



A operação de Projeção (π)

- A projeção seleciona as colunas específicas de uma relação
 - Ex: π nome, salario (**EMPREGADO**)
- > projeta apenas os dados das colunas nome e salário da relação empregado

nome	salario
Ana	5000
João	3000
Maria	7000

```
1 SELECT nome, salario
2 FROM empregado;
```




A operação de Junção (⋈)

- A junção combina informações de tabelas relacionadas utilizando uma condição
 - Ex: **EMPREGADO** ⋈ **DEPARTAMENTO**

id	nome	salario	id_departamento
1	Ana	5000	10
2	João	3000	20

id	nome
10	TI
20	RH



```
1 SELECT e.nome,  
2       d.nome  
3 FROM empregado e  
4 INNER JOIN departamento d  
5     ON e.numero_departamento = d.numero;
```

nome	nome
Ana	TI
João	RH



Resumo

```
1 SELECT nome
2 FROM empregado
3 WHERE salario > 3000;
```

- Visão do usuário:
 - mostre nomes de empregados que ganham mais que 3000
- Visão do banco
 - [1] filtrar empregados com salário maior que 3000
 - [2] selecionar apenas os nomes
 - [3] álgebra relacional:
 - π nome (σ salario > 3000 (EMPREGADO))



Desempenho

- Imagine que a tabela **EMPREGADO** contém 10 milhões de registros, como o banco vai encontrar o funcionário com o cpf "111.111.111-11" caso o cpf não seja uma chave primária?

```
1 SELECT *  
2 FROM empregado  
3 WHERE cpf = '111.111.111-11';
```

- Linha por Linha?

VS

> depende

- uma estrutura otimizada de busca?

Índices



- Um índice é uma estrutura criada para acelerar buscas
 - Em um livro sem índice precisamos percorrer todas as páginas para encontrar algo
 - Com índice encontramos a página que queremos no índice e então vamos diretamente para ela

```
1 CREATE INDEX idx_cpf  
2 ON empregado(cpf);
```

- Sem índices a nossa consulta precisaria fazer um SCAN ($O(n)$) até encontrar o que buscamos, com o índice temos uma busca otimizada ($O(\log n)$) por utilizar uma B-Tree como estrutura auxiliar
 - Porém, aumentam o custo de escrita (devo escrever em dois lugares) e ocupam espaço adicional (trade-offs)

Índices



- Chaves primárias são indexadas automaticamente na maioria dos SGBDs relacionais
- Índices devem ser usados quando:
 - buscas são mais frequentes que inserções e atualizações
 - colunas são usadas com **WHERE** e/ou **JOIN**

```
1 CREATE INDEX idx_cpf  
2 ON empregado(cpf);
```

- Devem ser evitados em tabelas pequenas, colunas que são raramente utilizadas, ou em colunas com muitos valores repetidos.
 - Ex: Coluna sexo: M, F



Otimizador de Consultas

- O otimizador é o componente responsável por:
 - analisar consultas, estimar custos e escolher estratégias de execução
- O banco pode decidir entre SCAN completo, uso de índice, diferentes tipos de JOIN
 - A ideia é executar a consulta da forma mais eficiente possível
 - Nesse processo a consulta pode ser reconstruída, trazendo os mesmos resultados de uma forma mais otimizada



Paginação

- Na tabela empregado de 10 milhões de registros é pouco provável que consigamos carregar todos eles em tela após uma consulta:
 - vai faltar memória, causar lentidão e prejudicar a experiência do usuário

```
1 SELECT *  
2 FROM empregado;
```

- A paginação é o processo de dividir um grande conjunto de dados em páginas (partes menores do todo)
- Sites de compras geralmente fazem isso:
 - 10, 20, 50 produtos por página



Paginação no SQL - **LIMIT** e **OFFSET**

- **LIMIT** nos diz quantas linhas serão retornadas
- **OFFSET** quantas linhas queremos pular
 - Ex:
 - Pular 20 registros e retornar os próximos 10.
 - Trazer somente os 5 primeiros
 - Pular os 5 primeiros e trazer os 5 subsequentes



```
1 SELECT *  
2 FROM empregado  
3 ORDER BY cpf  
4 LIMIT 10 OFFSET 20;
```



```
1 SELECT *  
2 FROM empregado  
3 ORDER BY cpf  
4 LIMIT 5 OFFSET 0;
```

carlos.melo@upe.br



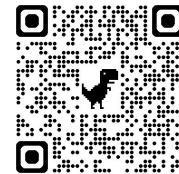
```
1 SELECT *  
2 FROM empregado  
3 ORDER BY cpf  
4 LIMIT 5 OFFSET 5;
```




Plano de execução

- O plano de execução define:
 - como buscar dados
 - quais índices utilizar (se houver mais de um)
 - qual JOIN executar
 - ordem das operações

```
1 EXPLAIN
2 SELECT *
3 FROM empregado
4 WHERE cpf = '111.111.111-11';
```



Prof. Dr. Carlos Melo
<https://calendly.com/prof-casm>

 carlos.melo@upe.br